import the requaried libraries

In [ ]:
```
import tensorflow as tf
from tensorflow.keras import layers, models, datasets
import matplotlib.pyplot as plt
```

pre-process the data

In [ ]:
```
def load_data():
  (x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
  x_train, x_test = x_train / 255.0, x_test / 255.0 # Normalize to [0, 1]
  y_train = tf.keras.utils.to_categorical(y_train, 10) # One-hot encoding
  y_test = tf.keras.utils.to_categorical(y_test, 10)
  return (x_train, y_train), (x_test, y_test)
```

Define the CNN model

In [ ]:
```
def create_model():
  model = models.Sequential()
  model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32,
  model.add(layers.MaxPooling2D((2, 2)))
  model.add(layers.Conv2D(64, (3, 3), activation='relu'))
  model.add(layers.MaxPooling2D((2, 2)))
  model.add(layers.Conv2D(64, (3, 3), activation='relu'))
  model.add(layers.Flatten())
  model.add(layers.Dense(64, activation='relu'))
  model.add(layers.Dense(10, activation='softmax')) # Output layer for 10
  return model
```

Compile the model

In [ ]:
```
def compile_model(model):
  model.compile(optimizer='adam',
    loss='categorical_crossentropy',
                metrics=['accuracy'])
```

train the model

In [ ]:
```
def train_model(model, x_train, y_train, x_test, y_test):
  history = model.fit(x_train, y_train, epochs=10,
    validation_data=(x_test, y_test), batch_size=64)
  return history
```

training performance for visualization

In [ ]:
```
def plot_history(history):
```

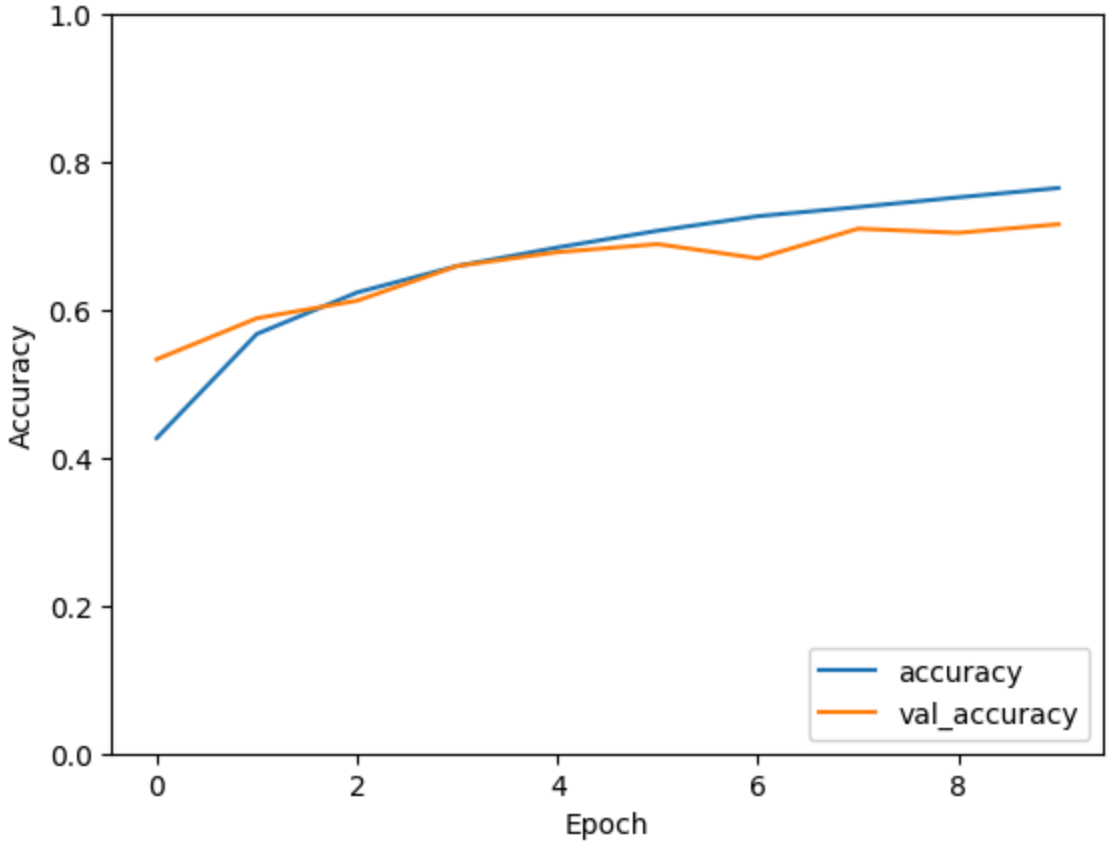
```
  plt.plot(history.history['accuracy'], label='accuracy')
  plt.plot(history.history['val_accuracy'], label='val_accuracy')
  plt.xlabel('Epoch')
  plt.ylabel('Accuracy')
  plt.ylim([0, 1])
  plt.legend(loc='lower right')
  plt.show()
```

main function

In [ ]:
```
if __name__ == "__main__":
  (x_train, y_train), (x_test, y_test) = load_data()
  model = create_model()
  compile_model(model)
  history = train_model(model, x_train, y_train, x_test, y_test)
  test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
  print(f"Test accuracy: {test_acc:.2f}")
  plot_history(history)
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 788s 5us/step
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 72s 90ms/step - accuracy: 0.4266 - loss: 1.5
666 - val_accuracy: 0.5332 - val_loss: 1.2898
Epoch 2/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 78s 85ms/step - accuracy: 0.5675 - loss: 1.2
142 - val_accuracy: 0.5887 - val_loss: 1.1457
Epoch 3/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 67s 85ms/step - accuracy: 0.6236 - loss: 1.0
699 - val_accuracy: 0.6125 - val_loss: 1.0854
Epoch 4/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 65s 83ms/step - accuracy: 0.6594 - loss: 0.9
716 - val_accuracy: 0.6594 - val_loss: 0.9700
Epoch 5/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 65s 83ms/step - accuracy: 0.6845 - loss: 0.9
025 - val_accuracy: 0.6782 - val_loss: 0.9285
Epoch 6/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 65s 83ms/step - accuracy: 0.7071 - loss: 0.8
379 - val_accuracy: 0.6890 - val_loss: 0.9048
Epoch 7/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 65s 83ms/step - accuracy: 0.7268 - loss: 0.7
878 - val_accuracy: 0.6698 - val_loss: 0.9745
Epoch 8/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 81s 83ms/step - accuracy: 0.7392 - loss: 0.7
514 - val_accuracy: 0.7099 - val_loss: 0.8600
Epoch 9/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 69s 88ms/step - accuracy: 0.7523 - loss: 0.7
100 - val_accuracy: 0.7043 - val_loss: 0.8786
Epoch 10/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 78s 82ms/step - accuracy: 0.7649 - loss: 0.6
761 - val_accuracy: 0.7159 - val_loss: 0.8490
313/313 - 4s - 11ms/step - accuracy: 0.7159 - loss: 0.8490
Test accuracy: 0.72
```


1.0
0.8
0.6
0.4
0.2
0.0
Accuracy
0
2
4
6
8
Epoch
accuracy
val_accuracy